TASK 3

Secure Coding Review

Security audit of a Python (Flask) web application

CodeAlpha Cyber Security Internship

Submitted by: [Your Name] | Intern ID: [Your ID] | Date: [Date]

1. Executive Summary

This report documents a security code review of a small Python Flask web application containing user login, search, file download, network ping and data-loading features. The review combined **manual code inspection** with guidance from the **OWASP Top 10** and **CWE** catalogue, and recommends running **Bandit** (Python static analyzer) to confirm findings automatically.

The review identified **8 vulnerabilities: 5 Critical, 2 High and 1 Medium**. Several allow full server compromise (remote code execution) by an unauthenticated attacker. All issues have straightforward fixes, shown in Section 6.

Severity	Count	Findings
Critical	5	SQL injection, command injection, insecure deserialization, path traversal, debug mode
High	2	Cross-site scripting / template injection, weak password hashing
Medium	1	Hard-coded secret key

2. Scope and Methodology

- **Language / framework:** Python 3, Flask, SQLite.
- **Application:** single-file demonstration web app (app.py, about 35 lines) written to include common real-world mistakes.
- **Techniques:** manual line-by-line review; data-flow tracing from user input (sources) to dangerous functions (sinks); mapping to OWASP Top 10 (2021) and CWE.
- **Recommended tooling:** Bandit (`pip install bandit; bandit -r .`), Semgrep, pip-audit for dependency checks.
- **Severity** is rated by exploitability and impact (Critical / High / Medium / Low).

3. Code Under Review

The numbers in comments mark each finding discussed below.

```
import sqlite3, hashlib, os, pickle, subprocess
from flask import Flask, request, render_template_string, send_file

app = Flask(__name__)
app.secret_key = "admin123"                      # (1) hard-coded secret
DB = "users.db"

@app.route("/login", methods=["POST"])
def login():
    u = request.form["username"]
    p = hashlib.md5(request.form["password"].encode()).hexdigest()   # (2) weak hash
    q = "SELECT * FROM users WHERE name='%s' AND pw='%s'" % (u, p)   # (3) SQL injection
    row = sqlite3.connect(DB).execute(q).fetchone()
    return "Welcome " + u if row else "Invalid"

@app.route("/search")
def search():
    term = request.args.get("q", "")
    return render_template_string("<h1>Results for " + term + "</h1>")  # (4) XSS / SSTI

@app.route("/download")
def download():
    return send_file("uploads/" + request.args["file"])          # (5) path traversal

@app.route("/ping")
```

```
def ping():
    host = request.args["host"]
    return subprocess.getoutput("ping -c 1 " + host)             # (6) command injection

@app.route("/load", methods=["POST"])
def load():
    return str(pickle.loads(request.data))                       # (7) insecure deserialization

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)                          # (8) debug mode exposed
```

4. Summary of Findings

#	Vulnerability	Location	Severity	CWE	OWASP 2021
1	Hard-coded secret key	app.secret_key	Medium	CWE-798	A05 Misconfiguration
2	Weak password hashing (MD5, unsalted)	/login	High	CWE-328 / 916	A02 Cryptographic Failures
3	SQL injection	/login	Critical	CWE-89	A03 Injection
4	XSS / template injection	/search	High	CWE-79 / 1336	A03 Injection
5	Path traversal	/download	Critical	CWE-22	A01 Broken Access Control
6	OS command injection	/ping	Critical	CWE-78	A03 Injection
7	Insecure deserialization (pickle)	/load	Critical	CWE-502	A08 Integrity Failures
8	Debug mode + bind to 0.0.0.0	app.run	Critical	CWE-489	A05 Misconfiguration

5. Detailed Findings and Remediation

Finding 3: SQL Injection (Critical) - /login

Issue: User input is concatenated directly into the SQL string. An attacker can submit the username `admin' --` to comment out the password check and log in as any user, or use UNION queries to dump the database.

Fix: Use parameterized queries (placeholders) so input is always treated as data: `execute("SELECT pw FROM users WHERE name = ?", (u,))`. Use an ORM such as SQLAlchemy and apply least-privilege DB accounts.

Finding 6: OS Command Injection (Critical) - /ping

Issue: The host parameter is appended to a shell command. Input such as `8.8.8.8; cat /etc/passwd` executes arbitrary commands with the server's privileges.

Fix: Avoid the shell: use `subprocess.run([...list...])` without shell=True, validate input against a strict allow-list (hostname/IP regex), set timeouts, or use a library instead of calling system tools.

Finding 7: Insecure Deserialization (Critical) - /load

Issue: `pickle.loads` on untrusted data executes attacker-crafted objects during unpickling, giving remote code execution.

Fix: Never unpickle untrusted data. Use JSON (or another data-only format) and validate with a schema (e.g., pydantic, jsonschema). Sign data with HMAC if integrity is needed.

Finding 5: Path Traversal (Critical) - /download

Issue: The filename is joined to a directory without validation. Requesting `?file=../../etc/passwd` reads arbitrary files, including source code and secrets.

Fix: Use `send_from_directory` (which rejects traversal), or resolve the path with os.path.realpath and verify it starts with the allowed directory. Prefer mapping IDs to files instead of accepting names.

Finding 8: Debug Mode and Public Binding (Critical) - app.run

Issue: Flask debug mode exposes the Werkzeug interactive debugger, which can allow remote code execution, and leaks stack traces. Binding to 0.0.0.0 exposes the dev server on all interfaces.

Fix: Set debug=False in production, run behind a production WSGI server (gunicorn/uWSGI) and reverse proxy (nginx) with HTTPS, and bind to localhost or a private interface.

Finding 4: Cross-Site Scripting / Server-Side Template Injection (High) - /search

Issue: User input is concatenated into the template string passed to render_template_string. Injecting `<script>` runs JavaScript in victims' browsers, and Jinja expressions like `{{7*7}}` are evaluated, which can escalate to code execution.

Fix: Never build templates from user input. Pass data as template variables (`{{ t }}` is auto-escaped), add a Content-Security-Policy header, and set cookies HttpOnly and Secure.

Finding 2: Weak Password Hashing (High) - /login

Issue: MD5 is fast and unsalted, so leaked hashes can be cracked in seconds using rainbow tables or GPUs.

Fix: Use a slow, salted password hash: `werkzeug.security.generate_password_hash` (scrypt/PBKDF2) or argon2/bcrypt. Add rate limiting and account lockout, plus MFA.

Finding 1: Hard-coded Secret Key (Medium) - app.secret_key

Issue: The secret signs session cookies. A weak, guessable value committed to source control lets attackers forge sessions.

Fix: Generate a long random key (`secrets.token_hex(32)`), load it from an environment variable or secrets manager, rotate regularly, and keep it out of Git (use .gitignore and secret scanning).

6. Remediated Code

The corrected version of the application applying all fixes above:

```
import os, sqlite3, secrets, re, subprocess, json
from flask import Flask, request, abort, render_template_string, send_from_directory
from markupsafe import escape
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.secret_key = os.environ["SECRET_KEY"]                # from environment / vault
UPLOAD_DIR = os.path.abspath("uploads")

@app.route("/login", methods=["POST"])
def login():
    u = request.form.get("username", "")
    p = request.form.get("password", "")
    row = sqlite3.connect("users.db").execute(
        "SELECT pw FROM users WHERE name = ?", (u,)).fetchone()    # parameterized
    if row and check_password_hash(row[0], p):                     # salted hash (scrypt/pbkdf2)
        return "Welcome " + str(escape(u))
    return "Invalid credentials", 401

@app.route("/search")
def search():
    term = request.args.get("q", "")
    return render_template_string("<h1>Results for {{ t }}</h1>", t=term)  # auto-escaped

@app.route("/download")
def download():
    return send_from_directory(UPLOAD_DIR, request.args.get("file", ""))   # blocks ../

@app.route("/ping")
def ping():
    host = request.args.get("host", "")
    if not re.fullmatch(r"[A-Za-z0-9.\-]{1,253}", host):
        abort(400)
    out = subprocess.run(["ping", "-c", "1", host], capture_output=True,
                         text=True, timeout=5)                     # no shell, list args
    return out.stdout

@app.route("/load", methods=["POST"])
def load():
    return json.dumps(json.loads(request.data))             # safe data format, not pickle

if __name__ == "__main__":
    app.run(host="127.0.0.1", debug=False)
```

7. Additional Observations

- **No authentication/authorization** on /download, /ping and /load: add login checks and role-based access control.
- **No CSRF protection:** use Flask-WTF CSRFProtect for state-changing forms.
- **No rate limiting:** use Flask-Limiter to slow brute force and DoS.
- **No security headers:** add CSP, X-Content-Type-Options, X-Frame-Options and HSTS (e.g., Flask-Talisman).
- **No logging/monitoring** of security events: log failed logins and errors (without logging secrets).
- **Dependency risk:** pin versions and scan with pip-audit or Dependabot.

8. Secure Coding Best Practices

Principle	Practice
Validate input	Treat all input as untrusted. Use allow-lists, type checks and length limits on the server.
Encode output	Escape data for its context (HTML, SQL, shell, URL). Use framework auto-escaping.
Parameterize	Never build SQL, shell commands or templates by string concatenation.
Least privilege	Run the app and DB accounts with only the permissions required.
Protect secrets	No credentials in code. Use environment variables or a secrets manager.
Use strong crypto	Standard libraries only; never invent your own; slow salted hashes for passwords; TLS everywhere.
Fail securely	Generic error messages to users; detailed logs for developers; debug off in production.
Defense in depth	Layer WAF, headers, rate limiting, monitoring and MFA.
Keep updated	Patch the framework and libraries; scan dependencies continuously.
Review and test	Code review, SAST (Bandit/Semgrep), DAST (OWASP ZAP), and unit tests for security cases.

9. Verification Commands

```
# Static analysis
pip install bandit
bandit -r app.py

# Example payloads used to confirm the issues (on your own test instance only)
curl -X POST -d "username=admin' --&password=x" http://127.0.0.1:5000/login
curl "http://127.0.0.1:5000/ping?host=8.8.8.8;id"
curl "http://127.0.0.1:5000/download?file=../app.py"
curl "http://127.0.0.1:5000/search?q={{7*7}}"     # returns 49 if vulnerable
```

Note: Test payloads only against your own local test application, never against systems you do not own or have permission to test.

10. Conclusion

The original application was unsafe for deployment: an unauthenticated attacker could read files, take over accounts and execute commands. The root causes are common: trusting user input, building commands/queries by string concatenation, weak cryptography and insecure configuration. Applying parameterization, strict validation, safe APIs, strong hashing and hardened configuration removes every identified issue. Secure coding should be combined with automated scanning in CI/CD and regular peer

review.

References: OWASP Top 10 (2021), OWASP Cheat Sheet Series, CWE Top 25, Flask security documentation, Bandit documentation.